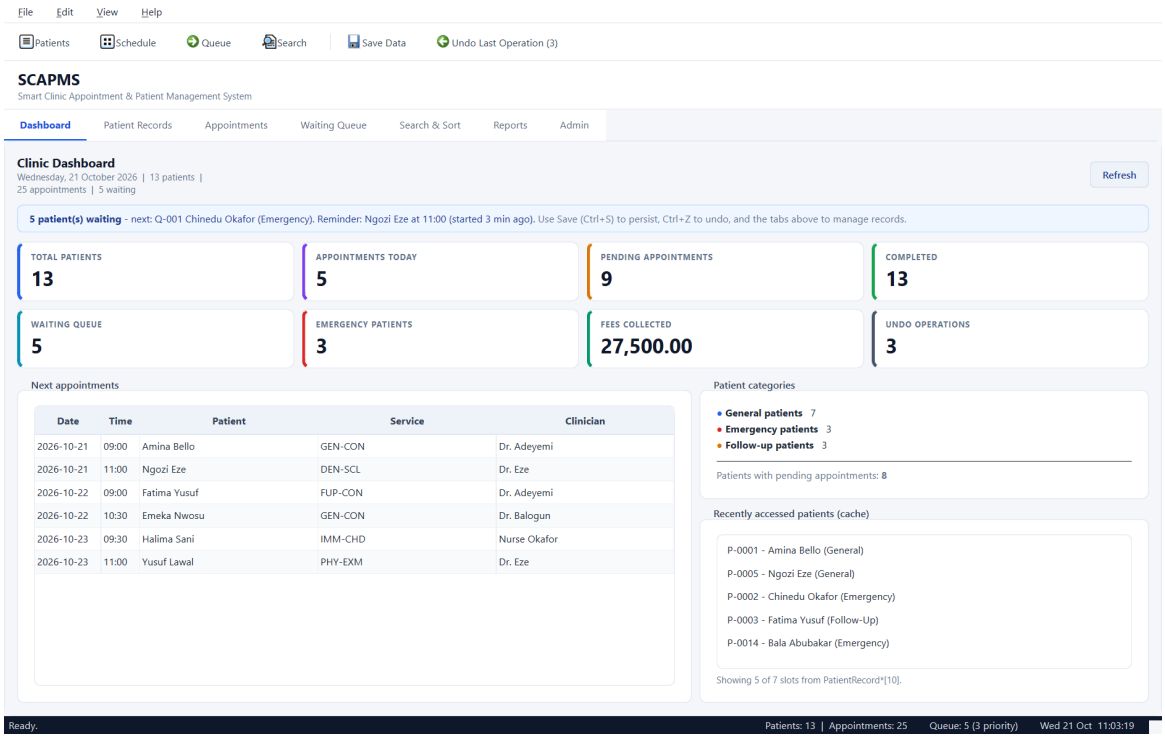
COS202 PROGRAMMING II

LAB ASSESSMENT PROJECT

SCAPMS

Smart Clinic Appointment & Patient Management System



Submitted by: Otito Ugonna Emeka-Ebigbo (GitHub: Razznik99)

Email: o.emeka-ebigbo9627@miva.edu.ng

Student ID: 301819627

Matriculation Number: 2025/A/SENG/0253

Department: Software Engineering

Repository: https://github.com/Razznik99/SCAPMS-COS202

Date: 21 October 2026

C++17 · Qt 6.8 Widgets · 279 automated checks

Contents

1 Project Description **1**

2 System Features **1**

3 Class Hierarchy **1**

4 Data Structures Used **2**

5 Searching Algorithms **2**

6 Sorting Algorithms **2**

7 Recursive Algorithm **3**

8 GUI Design **3**

9 Exception Handling **4**

10 Algorithms Chosen and Reasons **4**

11 Challenges Encountered **4**

12 How the Challenges Were Resolved **5**

13 Verification Summary **5**

14 Domain Class Diagram **6**

15 Layered Architecture **7**

16 Main Dashboard **8**

17 Patient Registration and Records **9**

18 Appointment Management **10**

19 Search and Sort Interface **11**

20 Waiting Queue **12**

21 Reports **13**

22 Administration and Undo **14**

23 Repository and Organisation **15**

24 Core Code Listings **15**

25 Requirement Traceability **22**

26 Build and Run **22**

27 Test Evidence **22**

28 Data File Formats **23**

Submission Checklist **24**

PART 1
Project Report

1 Project Description

SCAPMS (Smart Clinic Appointment & Patient Management System) is a desktop application written in C++17 with the Qt 6 Widgets framework for a university clinic. It replaces the paper appointment book and patient register with one persistent workstation application: clinic staff register patients, book and cancel appointments, keep a triage-aware waiting queue, search and sort the register, undo administrative mistakes, and print simple reports.

The system is deliberately split into a pure C++ domain layer (model and controller) and a Qt presentation layer, so that the business rules can be tested without a display. Patient, appointment and service data are stored as CSV files, and the application seeds a realistic demonstration clinic on first run. The complete implementation, 279 automated checks and seven documentation screenshots are published at https://github.com/Razznik99/SCAPMS-COS202.

2 System Features

Feature	Code	Notes
Register / edit / delete patients	PatientPanel	Per-type fields, auto ID
Appointments	AppointmentPanel	Schedule, cancel, complete
Waiting queue	QueuePanel	FIFO plus priority lane
Search and sort	SearchPanel	Four searches, four sorts
Undo	AdminPanel	std::stack of commands
Reports	ReportPanel	Five reports, CSV export
Persistence	ClinicDatabase	Three CSV data files
Dashboard and timers	MainWindow	Live statistics, reminders
Import / export	AdminPanel	QFileDialog operations

3 Class Hierarchy

`PatientRecord` is the abstract base class. It stores the patient ID, name, age, contact details, gender and registration date, and it declares seven pure virtual operations: `category()`, `priorityLevel()`, `priorityLabel()`, `consultationFee()`, `clinicalSummary()`, `typeTag()` and `clone()`. Because these are pure virtual, no `PatientRecord` object can be created: only the three concrete subclasses can.

- **GeneralPatient** is a routine outpatient (fee 2,500, priority 3, “Normal”), carrying an insurance provider and a chronic condition.
- **EmergencyPatient** is a triage case (priority 1, “CRITICAL” or “HIGH”); its fee is 7,500 plus a triage surcharge, and it deliberately does **not** implement `Appointable` because emergencies are walk-in only.
- **FollowUpPatient** is a scheduled review (priority 2, “Scheduled”), knows its original visit date, review interval, due date and referring doctor, and receives a reduced fee when the review is booked within two weeks.

`Appointable` is the interface-like abstract class required by the brief. It declares `scheduleAppointment()`, `cancelAppointment()`, `appointmentIds()` and `appointmentCount()`. The reusable implementation lives in `AppointableImpl`, and `GeneralPatient` and `FollowUpPatient` inherit both `PatientRecord` and `AppointableImpl`. This multiple inheritance shows the difference between inheriting identity/state (`PatientRecord`) and inheriting a capability (`Appointable`).

Polymorphism is used throughout the GUI: the patient table is built from a `std::vector<PatientRecord*>` and calls the virtual `category()`, `priorityLabel()` and `consultationFee()` for each row, while the Admin panel can display the output of `describeAllPatients()` to prove that each call dispatches to the concrete type at run time.

Composition and association complete the model. `ClinicDatabase` **owns** the patient collection (an owning `std::vector` of `std::unique_ptr` records), the appointments and the service catalogue, so destroying the database destroys its records. `Appointment` is associated with one patient and one `MedicalService`, and a patient can be associated with many appointments through the `Appointable` bookkeeping. The full diagram is in Part 2.

4 Data Structures Used

Purpose	C++ type
Patient register (owned)	vector of unique_ptr
Appointments	vector of Appointment
Normal waiting lane	queue of QueueEntry
Priority waiting lane	queue of QueueEntry
Undo history	stack of unique_ptr
Recent-patient cache	PatientRecord* [10]
Dates, tickets, statuses	string, enum, struct

The register is a `vector` because patients are displayed, filtered and sorted as a sequence. The queue is a real `std::queue`: entries are pushed at the back and only popped from the front, which matches the clinic's first-come-first-served rule. Emergency cases use a second queue that is drained first, preserving FIFO within each lane. The undo history is a `std::stack` because the most recent operation must be reversed first (LIFO).

The cache is a raw fixed-size array rather than a container, as required. It stores non-owning `PatientRecord*` pointers in most-recently-used order; pointers remain valid because records are owned by `unique_ptr`, and every deletion removes the pointer from the array so it can never dangle.

5 Searching Algorithms

Search	Data state	Time	Function
Patient ID, linear	Unsorted	O(n)	linearSearchById
Name, binary	Sorted by name	O(log n)	binarySearchByName
ID, binary	Sorted by ID	O(log n)	binarySearchById
Name, recursive	Any order	O(n)	recursiveSearchByName
Category filter	Any order	O(n)	searchByCategory
Automatic	Inspected	O(log n) or O(n)	autoSearch

The binary search uses a hand-written lower-bound loop and refuses to run when the register is not sorted, telling the user to sort first; this makes the dependency between sorting and binary searching explicit. The Automatic mode inspects the data, uses binary search when the register is already sorted by name, falls back to a linear scan otherwise, and even performs an ID fallback when a sorted name index misses. Every result reports the number of comparisons and the elapsed microseconds in the GUI.

6 Sorting Algorithms

Algorithm	Average time	Extra space	Stable
Selection sort	O(n^2)	O(1)	No
Insertion sort	O(n^2)	O(1)	Yes
Merge sort	O(n log n)	O(n)	Yes
Quick sort	O(n log n)	O(log n)	No

All four are hand-written in `SortEngine.cpp`; no standard sorting function is used. Each algorithm can sort by patient name, patient ID, age, or the earliest pending appointment date (resolved through a callback supplied by `ClinicManager`). The GUI combo boxes select the algorithm and the field, and the result banner reports comparisons, element moves and elapsed time. Equal keys fall back to the patient name and ID, so all four algorithms produce the same total order and can be compared fairly. Selection sort minimises writes, insertion sort is adaptive and stable on nearly sorted data, merge sort guarantees O(n log n) and stability, and quick sort is the in-place choice when memory is tight.

7 Recursive Algorithm

Two recursive algorithms are implemented. The first is the recursive name search:

```
void SearchEngine::recursiveNameScan(const std::vector<model::PatientRecord*>& records,
                                     const std::string& fragment, std::size_t index,
                                     SearchResult& result) {
    if (index >= records.size()) {
        return; // base case
    }
    const auto* record = records[index];
    if (record != nullptr) {
        ++result.stats.comparisons;
        if (containsIgnoreCase(record->name(), fragment)) {
            result.matches.push_back(const_cast<model::PatientRecord*>(record));
        }
    }
    recursiveNameScan(records, fragment, index + 1, result); // recursive step
}
```

The base case is the index reaching the size of the register; each call processes exactly one record and delegates the rest to the next call. This supports partial matches (for example “bello” or “ami”), which the exact binary search cannot do.

The second is merge sort: `mergeSortRange()` recursively splits the range in half until the base case `left >= right`, then merges the sorted halves with a hand-written merge. The recursion depth is O(log n); the test suite sorts 72 records with every algorithm and checks the resulting order.

8 GUI Design

The main window is a `QTabWidget` with seven tabs: Dashboard, Patient Records, Appointments, Waiting Queue, Search & Sort, Reports, and Admin. A branded header, menu bar, toolbar and status bar frame the tabs, and the status bar permanently shows the patient count, queue length and a live clock.

Layout management uses six techniques and no absolute positioning:

Layout	Used for
QVBoxLayout	Panel roots and forms
QHBoxLayout	Toolbars and action rows
QGridLayout	Dashboard statistic cards

Layout	Used for
QFormLayout	Data-entry forms
QSplitter	Resizable panes
QStackedWidget	Type-specific fields

Event handling uses Qt signals and slots: button clicks (add, delete, schedule, cancel, search, sort, save, load), menu actions (New, Save, Load, Exit, Undo, Find, Refresh), text-field changes (live filter, ID validation, search preview, eligibility labels), and timer events. A one-second timer updates the clock, a thirty-second timer refreshes the dashboard and raises appointment reminders, and a fifteen-second timer refreshes queue waiting times and warns when a patient has waited more than twenty minutes.

Advanced GUI techniques include custom `PillDelegate` rendering that colours patient categories and appointment statuses, dynamic type-specific form pages, `QFileDialog` import and export, timers, validation dialogs, keyboard shortcuts (Ctrl+S, Ctrl+O, Ctrl+Z, Ctrl+F, Ctrl+Q, F5, Ctrl+1..7) and tooltips on every important control. A single Qt stylesheet (`resources/style.qss`) provides the visual theme. Part 3 contains the screenshots.

9 Exception Handling

The application defines a custom hierarchy rooted at `ClinicException : std::runtime_error`, with five concrete classes: `ValidationException`, `DuplicateIdException`, `NotFoundException`, `AppointmentException` and `FileException`. Each carries a short code and a human-readable message.

Validation happens close to the data: the model validates common fields and each subclass validates its own fields (age 0-120, names free of digits, contact digit counts, triage 1-5, follow-up dates not in the future). Controllers validate business rules (duplicate ID, non-bookable emergency patient, past appointment date, clinician double-booking, unknown patient or service, unsorted binary search, empty search input). GUI slots catch `ClinicException` and show a `QMessageBox`; a second catch of `std::exception` prevents an unexpected error from terminating the application. File loading is transactional: records are parsed into temporary containers and committed only when every line is valid, and a malformed line produces a `FileException` naming the file and line number. The automated test suite verifies every exception path.

10 Algorithms Chosen and Reasons

Linear search is the correct default for patient ID because the register is kept in registration order, not ID order; it needs no preprocessing and the ID is unique. **Binary search** is offered once the register has been sorted by name or ID, reducing the search to O(log n); the GUI therefore exposes sorting and searching side by side. **Recursive search** is the right tool for partial names, where the exact-match index cannot help. **Selection sort** is appropriate for a small, unsorted register because it minimises writes (O(n) swaps). **Insertion sort** is chosen after a previous sort because it is adaptive and stable. **Merge sort** is the safe general choice for large registers: O(n log n) with guaranteed stability. **Quick sort** is used when memory is tight and average-case speed matters, thanks to in-place partitioning with a median-of-three pivot. The GUI repeats this reasoning next to the combo boxes so the choice is visible to the user.

11 Challenges Encountered

1. **No C++ compiler or Qt installation** was present on the workstation.
2. **Cached raw pointers** could dangle after a patient was deleted.
3. **Undo of a cascaded delete** had to restore both the patient and the appointments that were removed with it.
4. **Binary search on unsorted data** would silently return wrong results.
5. The stylesheet appeared to be ignored: `MainWindow::styleSheet()` (the `QWidget` member) hid the free `styleSheet()` helper, returning an empty string.
6. GUI dialogs are modal, so they cannot be exercised by a plain console test.
7. Patient notes and addresses may contain commas and quotes, which would break naive CSV parsing.
8. A strict FIFO queue does not reflect real triage: an emergency patient must be seen first.

12 How the Challenges Were Resolved

1. Qt 6.8.3 and the MinGW 13.1 toolchain were installed with `aqtinstall`, and both qmake and CMake builds were verified with zero warnings.
2. The cache was implemented as `PatientRecord* recentPatients[10]` with explicit removal on deletion; the test suite deletes a cached patient and confirms the slot is cleared.
3. `DeletePatientCommand` stores a cloned patient and a copy of every removed appointment; undo re-inserts both and rebuilds the `Appointable` association.
4. `binarySearchByName()` checks `isSortedByName()` and throws a `ValidationException` explaining that sorting is required; the Automatic mode chooses the algorithm from the data state instead.
5. The helper was renamed to `appStyleSheet()`, which removed the name hiding and restored the theme.
6. Business rules were moved into the model and controller, which the 279-check console suite tests directly; the GUI dialogs were then verified with Windows UI Automation, which triggered the real "Patient name must be between 2 and 60 characters" dialog.
7. CSV fields are escaped on write and parsed with a quote-aware parser, and the round-trip is covered by the persistence tests.
8. Two queues are used: a priority lane for triage level 1 patients and a normal lane, with `callNextPatient()` draining priority first and preserving FIFO within each lane.

13 Verification Summary

The project builds with qmake (`mingw32-make`) and CMake (`cmake --build`) without warnings. `SCAPMS.exe --selftest` runs **279 checks** across twelve groups and reports `Passed: 279  Failed: 0`. The GUI was launched, all seven tabs were exercised, and the screenshot mode captured the live application. Full evidence is in Part 5.

PART 2
UML and Class Diagram

14 Domain Class Diagram

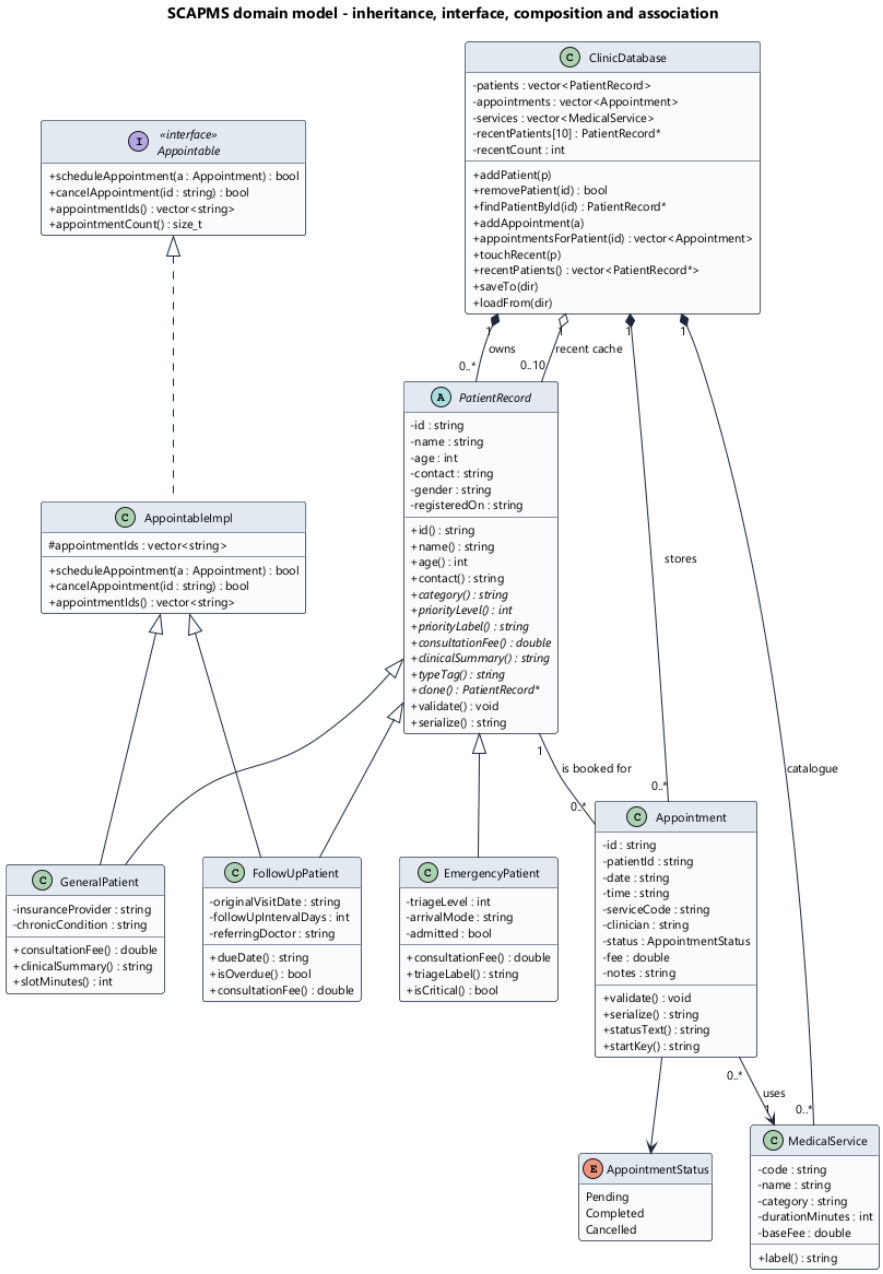


Figure 1: Domain class diagram: abstract base class, interface, inheritance, polymorphism, composition and association.

The diagram shows the four relationships required by the brief:

- **Inheritance** - `GeneralPatient`, `EmergencyPatient` and `FollowUpPatient` inherit from `PatientRecord`;

`GeneralPatient` and `FollowUpPatient` additionally inherit the `AppointableImpl` implementation.

- **Abstract classes** - `PatientRecord` and `Appointable` are abstract; their pure virtual operations are marked `{abstract}`.
- **Polymorphism** - all GUI table rows and the `describeAllPatients()` demonstration call the virtual operations through `PatientRecord*`.
- **Composition and association** - `ClinicDatabase` owns patients, appointments and services (filled diamonds), holds the non-owning recent-patient cache (hollow diamond), and `Appointment` references a `MedicalService` while a patient is associated with many appointments.

15 Layered Architecture

The GUI layer contains only presentation and event wiring. `ClinicManager` is the facade the panels call; it owns `ClinicDatabase`, `AppointmentController`, `ReportGenerator`, the queue lanes and the undo stack, and it delegates searching and sorting to stateless engines. The model and controller layers contain no Qt GUI types, which is why the console test suite can exercise the entire domain without a display.

The PlantUML sources are committed in the repository under `docs/uml/` (domain-class-diagram.puml and layered-architecture.puml).

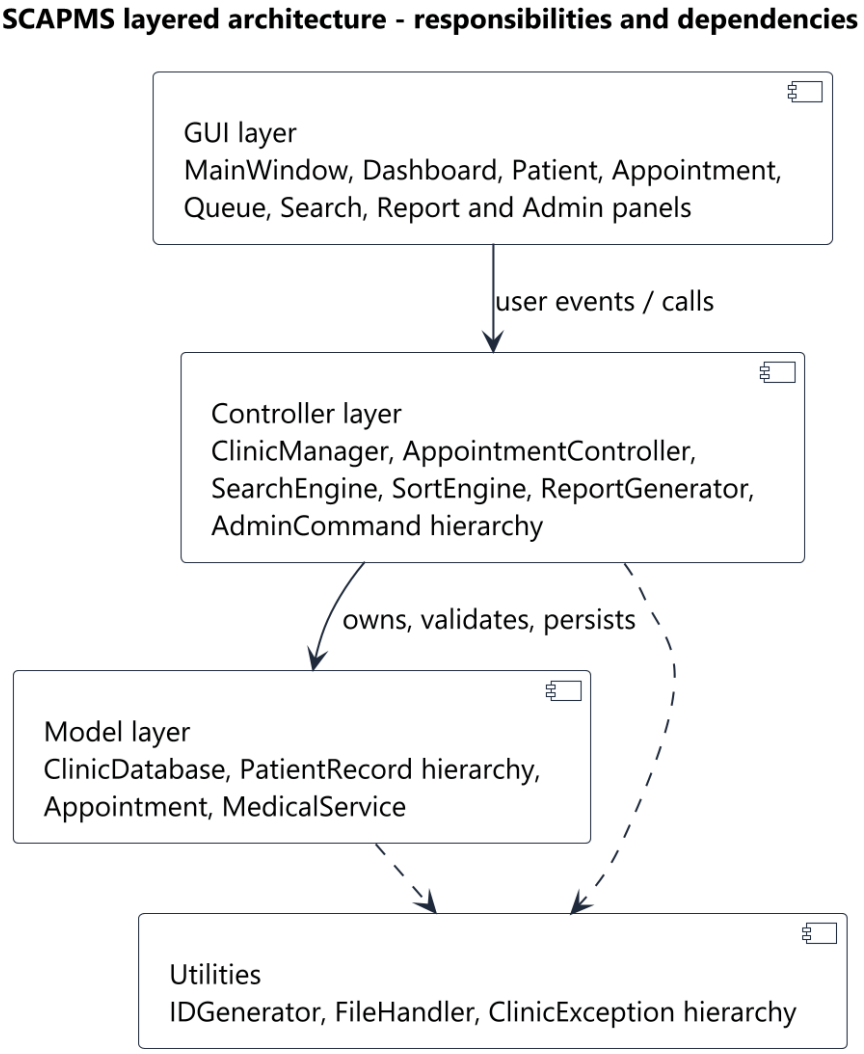


Figure 2: Layered architecture: the GUI depends on the controller facade, which owns the model and uses the utility classes.

PART 3
Screenshots

16 Main Dashboard

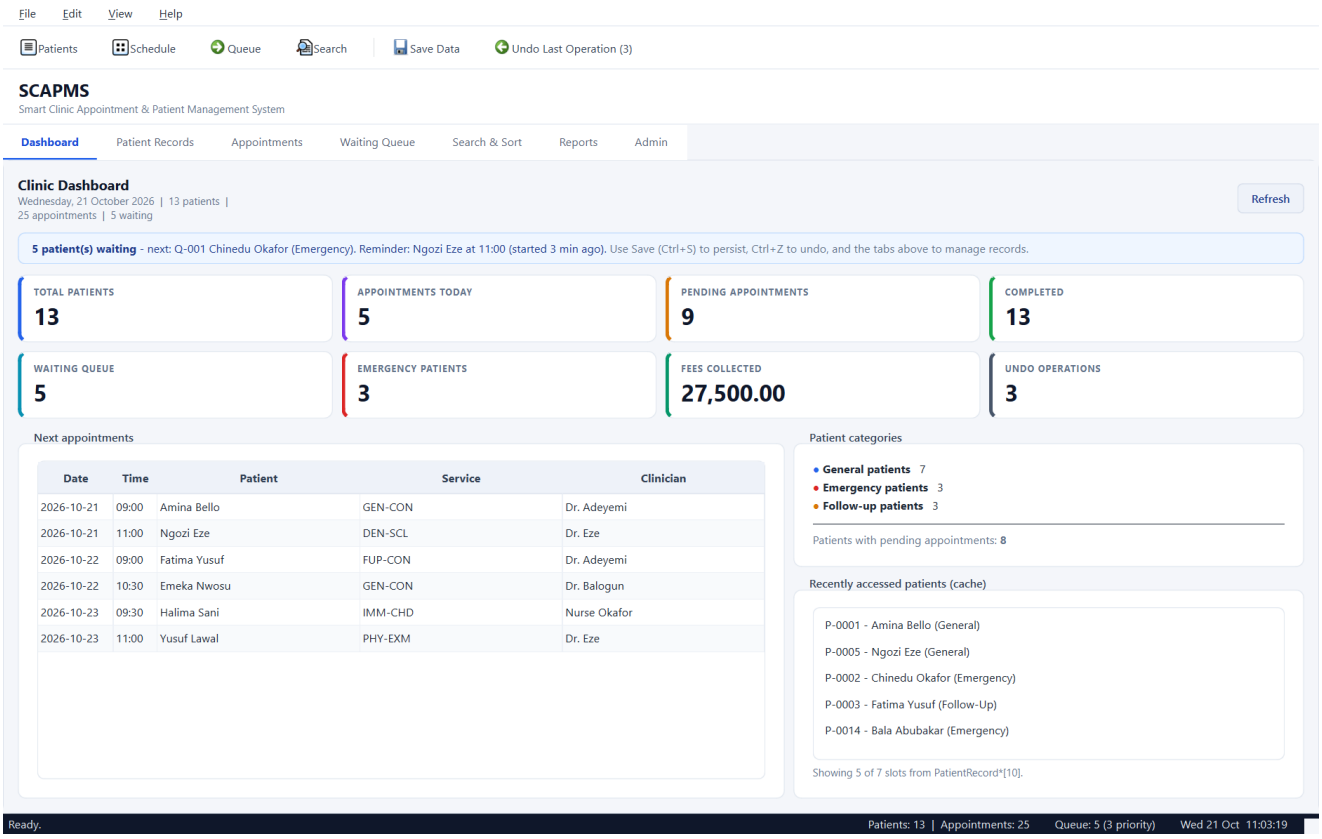


Figure 3: Live clinic dashboard with statistics, next appointments, category breakdown and recent-patient cache.

The dashboard aggregates the whole database: patient totals by category, appointments today, pending and completed counts, waiting queue length, fees collected and undo depth. It lists the next pending appointments, the category distribution and the five most recently accessed patients from the fixed-size cache, and its banner summarises the queue and the next appointment reminder.

17 Patient Registration and Records

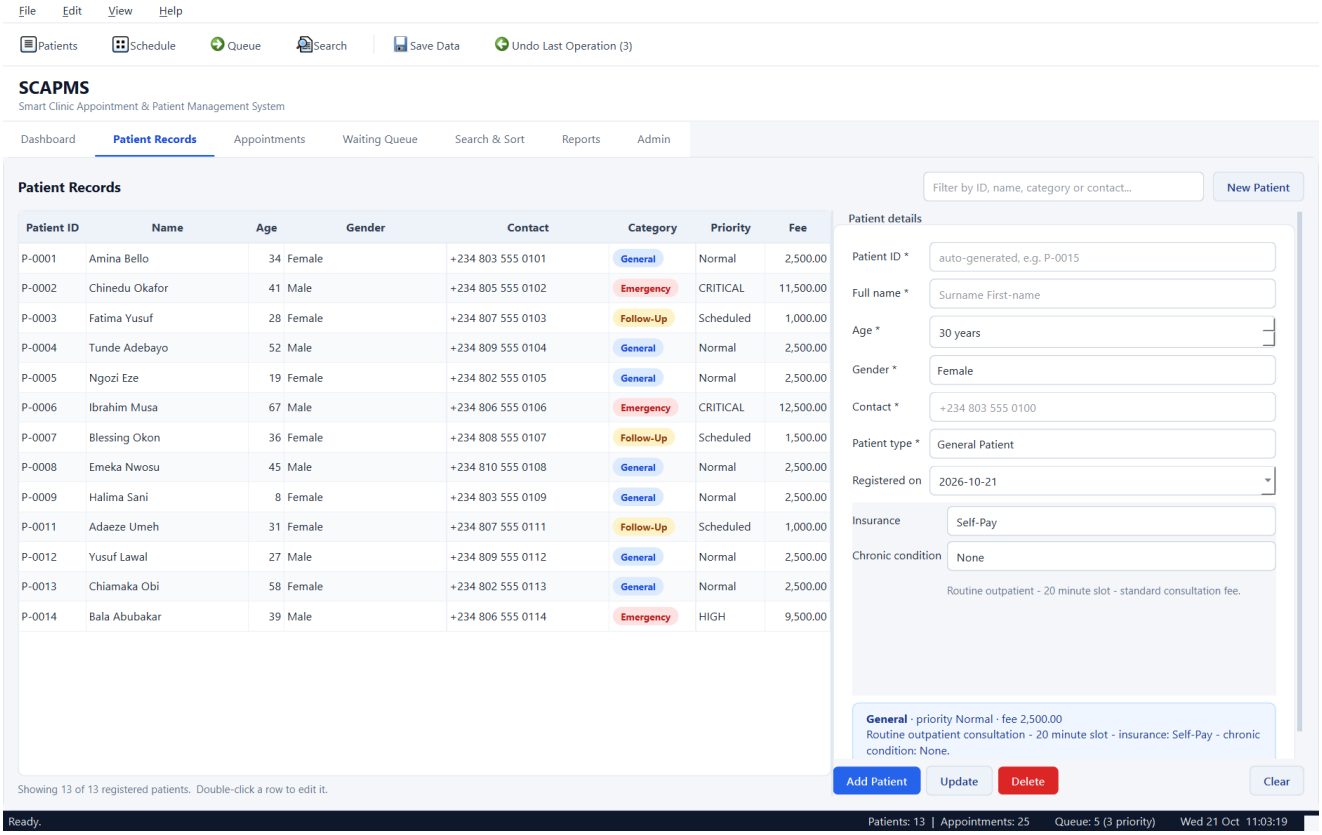


Figure 4: Patient records tab: colour-coded category pills, priority, fee, live filter and the type-specific registration form.

The left table shows every registered patient with colour-coded categories (General, Emergency, Follow-Up), priority and fee. Double-clicking a row loads it into the form for editing. The form's lower section changes with the patient type: insurance and chronic condition for General, triage level and arrival mode for Emergency, original visit date, interval and referring doctor for Follow-Up. The preview panel states the resulting category, priority, fee and clinical summary before the record is saved.

18 Appointment Management

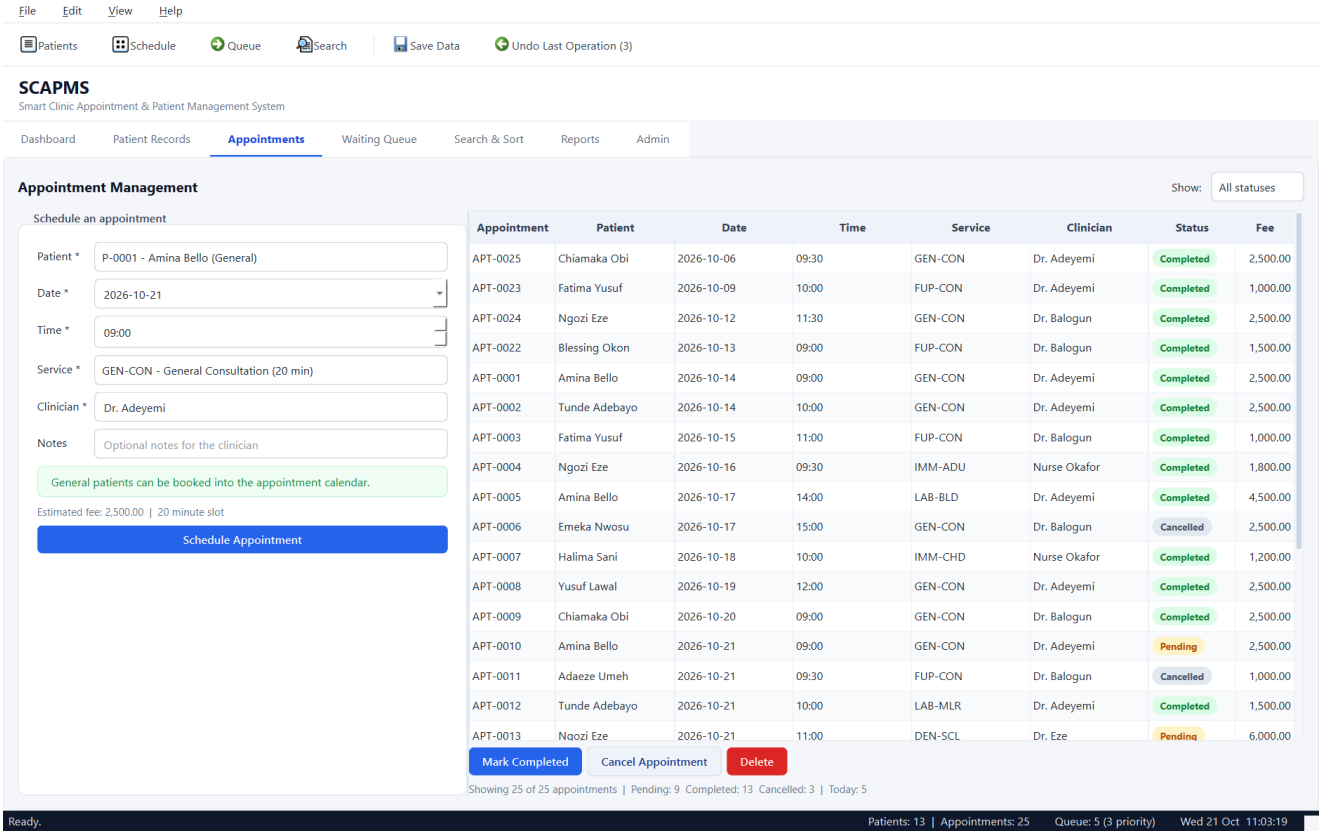


Figure 5: Appointment management: scheduling form with live eligibility and fee preview, and the appointment table with status pills.

Staff choose a patient, date, time, service and clinician; the panel previews the fee and states whether the patient type can be booked. Scheduling validates the date (no past dates), the time, the service, the clinician, clinician double-booking and a second active appointment in the same slot. The table shows every appointment with status pills (Completed, Pending, Cancelled) and the totals update after each operation.

19 Search and Sort Interface

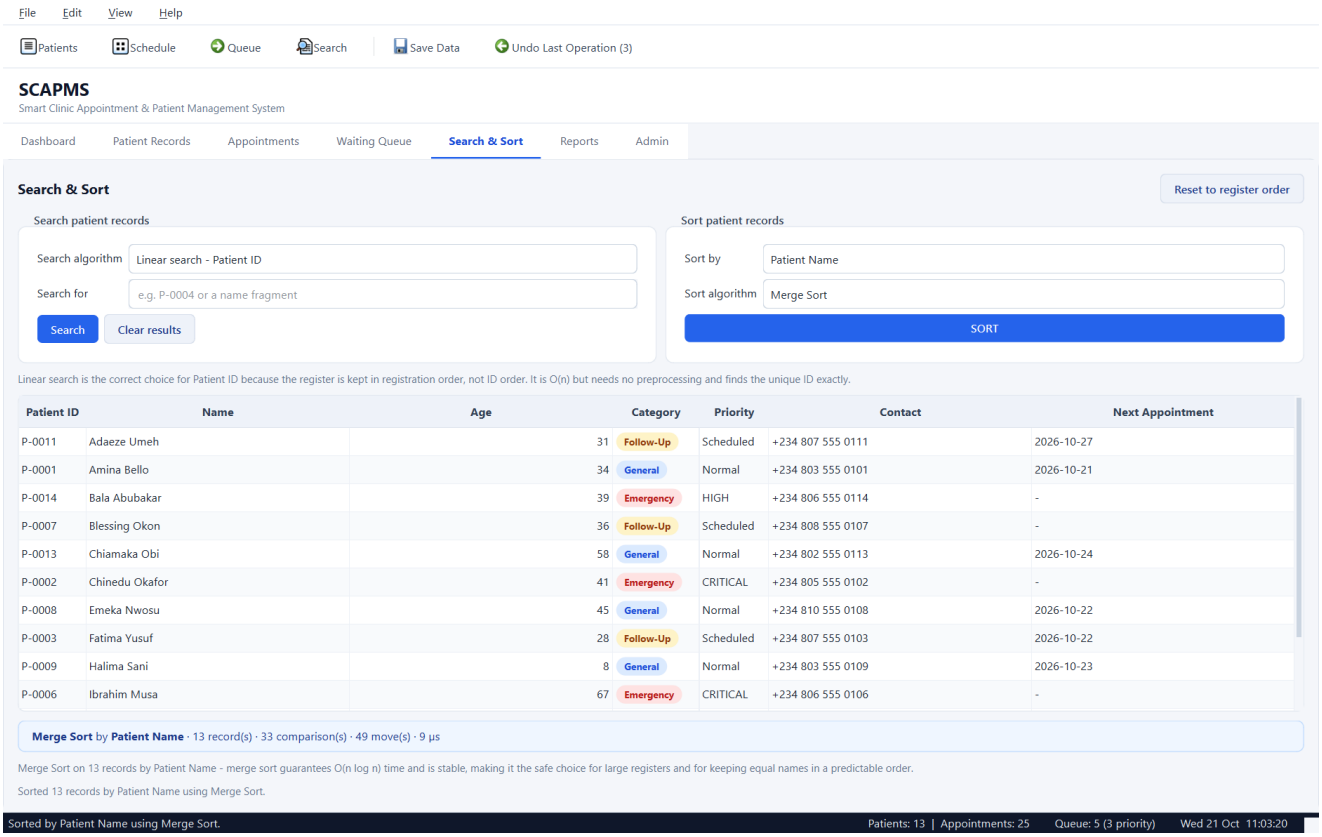


Figure 6: Search and sort tab: algorithm selectors, the sorted result, and the comparison/move/time statistics banner.

The Search group offers linear ID search, binary name search, recursive name search, category filtering and the data-aware Automatic mode. The Sort group offers four algorithms and four sort keys. The banner below the table reports the selected algorithm, the number of records, comparisons, moves and elapsed microseconds, and the rationale line explains why that algorithm suits the current data.

20 Waiting Queue

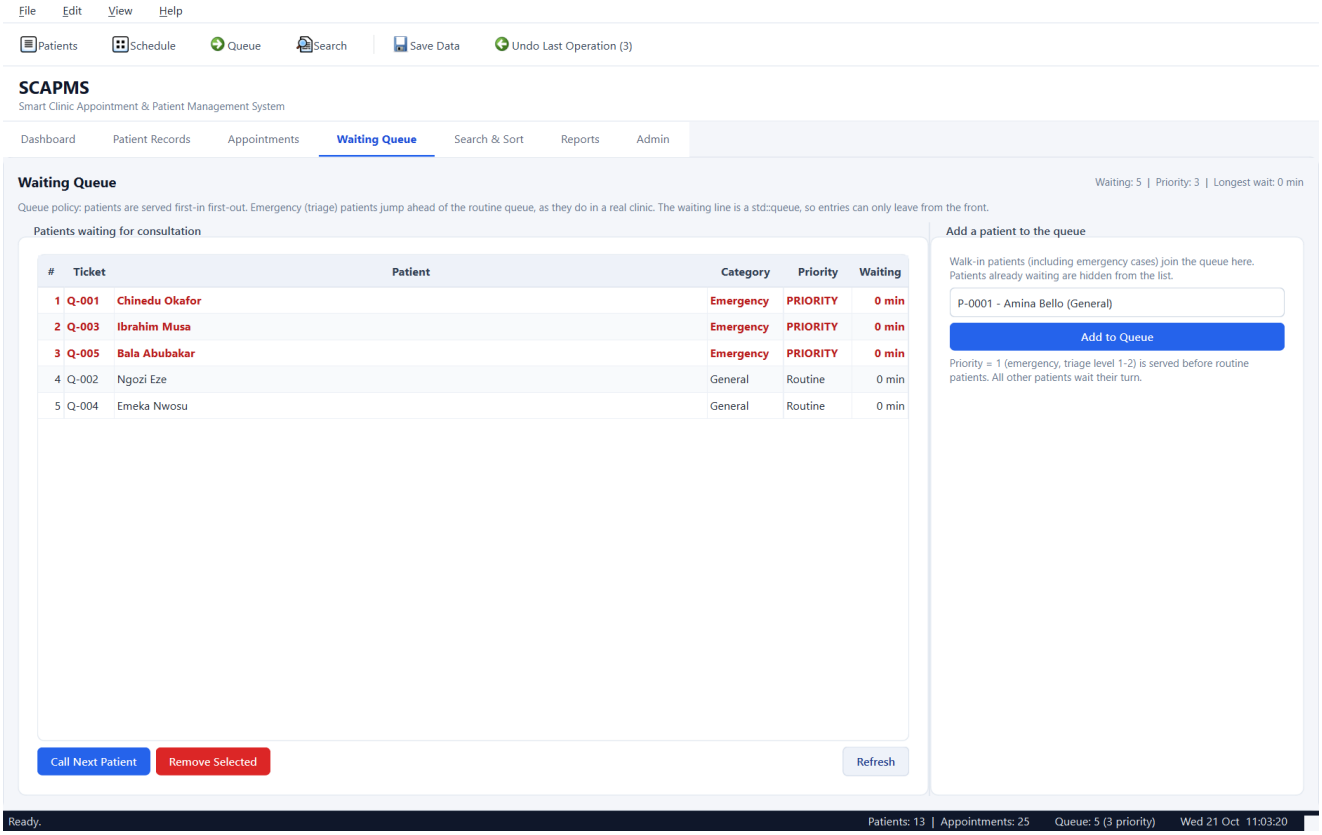


Figure 7: Waiting queue: priority and routine patients in ticket order with waiting times and queue actions.

The queue table lists the priority lane first and then the routine lane, in arrival order, with ticket numbers, categories, priority labels and waiting times. Emergency rows are shown in bold red. The timer refreshes waiting times every fifteen seconds and raises an alert when a patient has waited twenty minutes or more.

21 Reports

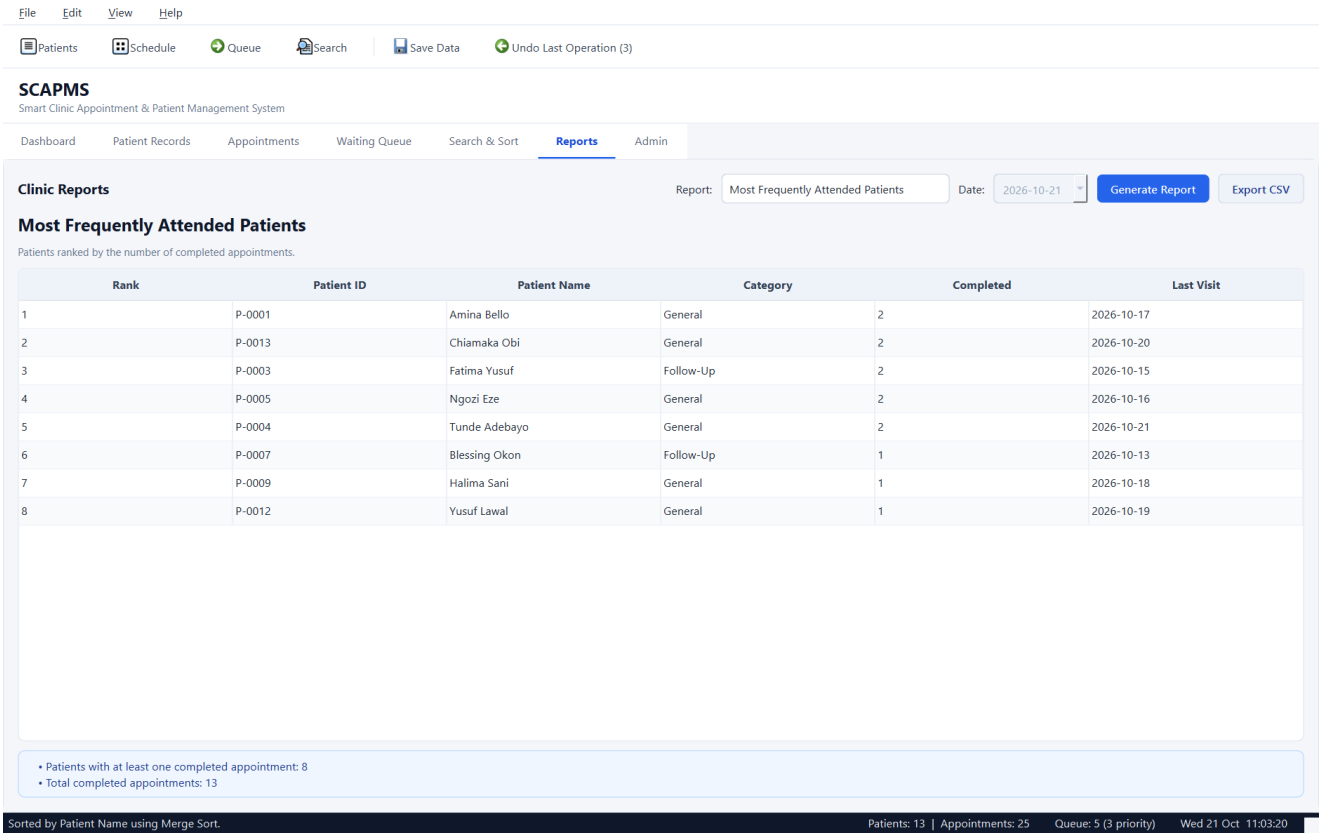


Figure 8: Most frequently attended patients report with the summary banner and CSV export button.

Five reports are available: most frequently attended patients, patients with pending appointments, category distribution, daily appointment summary, and revenue by medical service. Each report renders a table plus summary lines and can be exported to CSV through a file dialog.

22 Administration and Undo

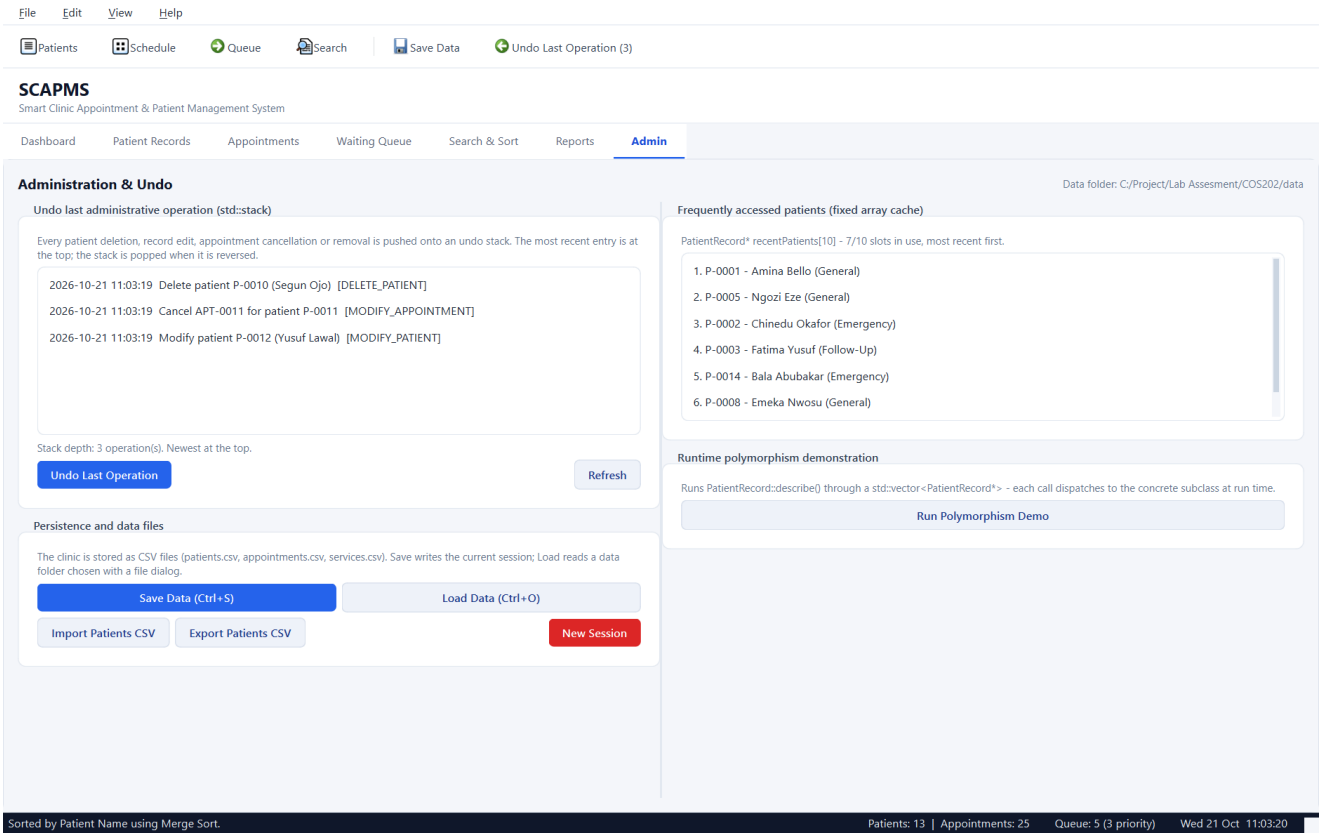


Figure 9: Administration tab: undo stack history, persistence controls, recent-patient cache and the polymorphism demonstration.

The Admin tab shows the undo stack with the newest operation first, the data-folder path, Save, Load, Import, Export and New Session controls, the ten-slot recent-patient cache and a button that runs the runtime-polymorphism demonstration. The screenshot was taken after three administrative operations (modify patient, cancel appointment, delete patient), which is why the stack depth is three and the patient count is thirteen.

PART 4
Source Code

23 Repository and Organisation

The complete source code, tests, resources, data files and documentation are published at:

GitHub: https://github.com/Razznik99/SCAPMS-COS202

Branch: `main` - initial commit `e98d680`

```
git clone https://github.com/Razznik99/SCAPMS-COS202.git
cd SCAPMS-COS202
```

The repository is organised into four source layers, a resource folder, a test harness, data files and documentation. The model, controller and utility layers are pure C++ and contain no Qt GUI types; the GUI layer converts to `QString` only at its boundary.

```
SCAPMS.pro                qmake project
CMakeLists.txt            CMake project
resources/                style.qss + resources.qrc
src/model/                PatientRecord hierarchy, Appointable, Appointment,
                          MedicalService, PatientFactory, ClinicDatabase
src/controller/           ClinicManager, AppointmentController, SearchEngine,
                          SortEngine, AdminCommand, ReportGenerator, DemoData
src/gui/                  MainWindow and seven panels, PillDelegate, UiHelpers
src/utils/                Exceptions, StringUtils, DateUtils, IDGenerator, FileHandler
tests/SelfTest.cpp        279-check console test suite
data/                     seed data files
screenshots/              seven documentation screenshots
docs/uml/                 PlantUML class diagrams
```

The implementation contains 62 C++ source/header files (8,265 lines) and a 712-line test harness. Every file starts with a commented header naming its layer and purpose.

24 Core Code Listings

The listings below are excerpts from the committed source; the GitHub repository contains the complete files.

24.1 Abstract base class (src/model/PatientRecord.h)

```
class PatientRecord {
public:
    PatientRecord(std::string id, std::string name, int age, std::string contact,
                  std::string gender, std::string registeredOn);

    virtual ~PatientRecord() = default;

    [[nodiscard]] const std::string& id() const noexcept { return m_id; }
    [[nodiscard]] const std::string& name() const noexcept { return m_name; }
    [[nodiscard]] int age() const noexcept { return m_age; }

    // ---- pure virtual operations (abstract class contract) ----------------
    [[nodiscard]] virtual std::string category() const = 0;
```

```
    [[nodiscard]] virtual int priorityLevel() const = 0;
    [[nodiscard]] virtual std::string priorityLabel() const = 0;
    [[nodiscard]] virtual double consultationFee() const = 0;
    [[nodiscard]] virtual std::string clinicalSummary() const = 0;
    [[nodiscard]] virtual std::string typeTag() const = 0;
    [[nodiscard]] virtual std::unique_ptr<PatientRecord> clone() const = 0;

    virtual void validate() const;
    [[nodiscard]] virtual std::string serialize() const;
};
```

24.2 Appointable interface (src/model/Appointable.h)

```
/// Pure interface: no data, only operations.
class Appointable {
public:
    virtual ~Appointable() = default;

    virtual bool scheduleAppointment(const Appointment& appointment) = 0;
    virtual bool cancelAppointment(const std::string& appointmentId) = 0;
    [[nodiscard]] virtual std::vector<std::string> appointmentIds() const = 0;
    [[nodiscard]] virtual std::size_t appointmentCount() const = 0;
};

class AppointableImpl : public Appointable {
public:
    bool scheduleAppointment(const Appointment& appointment) override;
    bool cancelAppointment(const std::string& appointmentId) override;
    [[nodiscard]] std::vector<std::string> appointmentIds() const override;
    [[nodiscard]] std::size_t appointmentCount() const override;
protected:
    std::vector<std::string> m_appointmentIds;
};
```

24.3 Type-specific behaviour (src/model/GeneralPatient.cpp)

```
int GeneralPatient::priorityLevel() const {
    return 3; // routine
}

double GeneralPatient::consultationFee() const {
    return 2500.0;
}

std::string GeneralPatient::clinicalSummary() const {
    return "Routine outpatient consultation - 20 minute slot - insurance: " +
           m_insuranceProvider + " - chronic condition: " + m_chronicCondition + ".";
}

std::string GeneralPatient::typeTag() const {
    return "GENERAL";
}

std::unique_ptr<PatientRecord> GeneralPatient::clone() const {
    return std::make_unique<GeneralPatient>(*this);
```

```
}
```

24.4 Linear search by patient ID (src/controller/SearchEngine.cpp)

```
for (const auto* record : records) {
    if (record == nullptr) {
        continue;
    }
    ++result.stats.comparisons;
    if (equalsIgnoreCase(record->id(), key)) {
        result.matches.push_back(const_cast<model::PatientRecord*>(record));
        break; // identifiers are unique
    }
}
```

24.5 Binary search lower bound (src/controller/SearchEngine.cpp)

```
std::size_t low = 0;
std::size_t high = records.size();
while (low < high) {
    const std::size_t middle = low + (high - low) / 2;
    ++comparisons;
    const std::string middleKey = toLower(records[middle]->name());
    if (middleKey < nameKey) {
        low = middle + 1;
    } else {
        high = middle;
    }
}
return low;
```

24.6 Binary search with sorted-data guard (src/controller/SearchEngine.cpp)

```
if (!isSortedByName(records)) {
    throw ValidationException(
        "Binary search needs data sorted by patient name. Sort the records first.",
        "search");
}

const std::size_t start = lowerBoundByName(records, key, result.stats.comparisons);
for (std::size_t i = start; i < records.size(); ++i) {
    ++result.stats.comparisons;
    if (toLower(records[i]->name()) != key) {
        break;
    }
    result.matches.push_back(const_cast<model::PatientRecord*>(records[i]));
}
```

24.7 Recursive name search (src/controller/SearchEngine.cpp)

```
void SearchEngine::recursiveNameScan(const std::vector<model::PatientRecord*>& records,
                                     const std::string& fragment, std::size_t index,
                                     SearchResult& result) {
```

```
    if (index >= records.size()) {
        return; // base case
    }
    const auto* record = records[index];
    if (record != nullptr) {
        ++result.stats.comparisons;
        if (containsIgnoreCase(record->name(), fragment)) {
            result.matches.push_back(const_cast<model::PatientRecord*>(record));
        }
    }
    recursiveNameScan(records, fragment, index + 1, result); // recursive step
}
```

24.8 Recursive merge sort (src/controller/SortEngine.cpp)

```
void mergeSortRange(std::vector<model::PatientRecord*>& records, std::size_t left,
                    std::size_t right, SortField field,
                    const SortEngine::DateLookup& lookup, SortStats& stats) {
    if (left >= right) {
        return;
    }
    const std::size_t middle = left + (right - left) / 2;
    mergeSortRange(records, left, middle, field, lookup, stats);
    mergeSortRange(records, middle + 1, right, field, lookup, stats);
    mergeHalves(records, left, middle, right, field, lookup, stats);
}
```

24.9 Fixed-size recent-patient cache (src/model/ClinicDatabase.cpp)

```
void ClinicDatabase::touchRecent(PatientRecord* patient) {
    if (patient == nullptr) {
        return;
    }

    int found = -1;
    for (int i = 0; i < m_recentCount; ++i) {
        if (m_recent[i] == patient ||
            (m_recent[i] != nullptr && m_recent[i]->id() == patient->id())) {
            found = i;
            break;
        }
    }

    const int limit = found >= 0 ? found
                                 : (m_recentCount < kRecentCapacity ? m_recentCount
                                                                    : kRecentCapacity - 1);
    for (int i = limit; i > 0; --i) {
        m_recent[i] = m_recent[i - 1];
    }
    m_recent[0] = patient;

    if (found < 0 && m_recentCount < kRecentCapacity) {
        ++m_recentCount;
    }
}
```

24.10 Waiting queue (src/controller/ClinicManager.cpp)

```
QueueEntry ClinicManager::joinQueue(const std::string& patientId) {
    model::PatientRecord* record = patient(patientId);
    if (record == nullptr) {
        throw NotFoundException("Patient", patientId);
    }
    if (isInQueue(patientId)) {
        throw ValidationException(record->name() + " is already waiting in the queue.");
    }

    QueueEntry entry;
    entry.patientId = record->id();
    entry.patientName = record->name();
    entry.category = record->category();
    entry.priority = record->priorityLevel();
    entry.priorityCase = record->priorityLevel() == 1;
    entry.joinedAt = scapms::utils::nowTimestamp();

    std::ostringstream ticket;
    ticket << "Q-" << std::setw(3) << std::setfill('0') << ++m_ticketCounter;
    entry.ticket = ticket.str();

    if (entry.priorityCase) {
        m_priorityQueue.push(entry);
    } else {
        m_normalQueue.push(entry);
    }
    return entry;
}
```

24.11 Undo of a cascaded delete (src/controller/ClinicManager.cpp)

```
void ClinicManager::deletePatient(const std::string& patientId) {
    model::PatientRecord* existing = m_database.findPatientById(patientId);
    if (existing == nullptr) {
        throw NotFoundException("Patient", patientId);
    }

    std::unique_ptr<model::PatientRecord> snapshot = existing->clone();
    std::vector<model::Appointment> removed =
        m_database.appointmentsForPatient(patientId);

    for (const auto& appointment : removed) {
        m_appointmentController.remove(appointment.id());
    }
    removeFromQueue(patientId);
    if (!m_database.removePatient(patientId)) {
        throw ClinicException("The patient record could not be removed.");
    }

    pushCommand(std::make_unique<DeletePatientCommand>(m_database, std::move(snapshot),
                                                       std::move(removed)));
}
```

24.12 Undo stack pop (src/controller/ClinicManager.cpp)

```
std::string ClinicManager::undoLastOperation() {
    if (m_undoStack.empty()) {
        throw ValidationException("There is no administrative operation to undo.");
    }
    std::unique_ptr<AdminCommand> command = std::move(m_undoStack.top());
    m_undoStack.pop();
    const std::string description = command->description();
    if (!command->undo()) {
        throw ClinicException("The last operation (" + description +
                              ") could not be reversed.");
    }
    return description;
}
```

24.13 CSV persistence (src/model/ClinicDatabase.cpp)

```
void ClinicDatabase::saveTo(const std::string& directory) const {
    FileHandler::ensureDirectory(directory);

    std::vector<std::string> patientLines;
    patientLines.push_back("# SCAPMS patient records (v1)");
    patientLines.push_back("#
↪   type,id,name,age,gender,contact,registeredOn,field1,field2,field3");
    for (const auto& patient : m_patients) {
        if (patient) {
            patientLines.push_back(patient->serialize());
        }
    }
    FileHandler::writeLines(patientsPath(directory), patientLines);

    // ... appointments.csv and services.csv are written the same way ...
}
```

24.14 GUI validation boundary (src/gui/PatientPanel.cpp)

```
try {
    auto patient = buildPatientFromForm(id);
    const QString label = toQt(patient->name()) + " (" + id + ")";
    m_manager->registerPatient(std::move(patient));
    m_editingPatientId.clear();
    refresh();
    onClearForm();
    emit dataChanged();
    emit statusMessage("Patient " + label + " registered.");
    QMessageBox::information(this, "Patient registered",
                             "Patient " + label + " was registered successfully.");
} catch (const scapms::utils::ClinicException& error) {
    QMessageBox::warning(this, "Invalid patient information", error.what());
} catch (const std::exception& error) {
    QMessageBox::critical(this, "Unexpected error", error.what());
}
```

24.15 Timers (src/gui/MainWindow.cpp)

```
// ---- timers (timer events) ---------------------------------------------
m_clockTimer = new QTimer(this);
m_clockTimer->setInterval(1000);
connect(m_clockTimer, &QTimer::timeout, this, &MainWindow::updateClock);
m_clockTimer->start();

m_refreshTimer = new QTimer(this);
m_refreshTimer->setInterval(30000);
connect(m_refreshTimer, &QTimer::timeout, this, &MainWindow::onTimerTick);
m_refreshTimer->start();
```

PART 5
Verification and Appendices

25 Requirement Traceability

Requirement	Implementation
Abstract class and pure virtuals	PatientRecord.h
Three patient subclasses	General / Emergency / Follow-Up
Appointable interface	Appointable.h
Runtime polymorphism	Calls through PatientRecord*
Composition	ClinicDatabase, Appointment
vector, queue, stack, array	Register, queues, undo, cache
Linear, binary, recursive search	SearchEngine
Four sorting algorithms	SortEngine
Events, menus, timers	MainWindow and seven panels
Six layout techniques	VBox, HBox, grid, form, splitter, stacked
Seven advanced GUI techniques	Delegate, dynamic fields, files, timers, dialogs, shortcuts, tooltips
Five exception classes	ClinicException hierarchy
CSV persistence	saveTo / loadFrom
Undo last operation	AdminCommand stack
Five simple reports	ReportGenerator

26 Build and Run

The application builds with Qt 6.8.3 and MinGW 13.1. The repository contains both build systems.

```
:: qmake
set PATH=C:\Qt\Tools\mingw1310_64\bin;C:\Qt\6.8.3\mingw_64\bin;%PATH%
mkdir build && cd build
qmake ..\SCAPMS.pro
mingw32-make -j4

:: CMake
cmake -S . -B build-cmake -G "MinGW Makefiles" ^
      -DCMAKE_PREFIX_PATH=C:/Qt/6.8.3/mingw_64
cmake --build build-cmake -j 4
```

Run the GUI with `SCAPMS.exe`, using `--data <folder>` to select a data folder. Run the tests with `SCAPMS.exe --selftest`; regenerate the documentation screenshots with `SCAPMS.exe --capture screenshots`.

27 Test Evidence

The console suite is organised into twelve groups: the patient hierarchy and validation; the `Appointable` interface and appointment rules; searching; sorting; the queue; undo; the cache; persistence; reports; statistics and polymorphism; string/date utilities; and the identifier generator. It prints a pass/fail line per check and returns the number of failures as its exit code.

```
SCAPMS self-test suite
-- 1. Patient hierarchy, validation and polymorphism --
-- 2. Appointable interface and appointment rules --
-- 3. Searching algorithms --
-- 4. Sorting algorithms (four algorithms, four keys) --
-- 5. Waiting queue (std::queue) --
-- 6. Undo stack for administrative operations --
-- 7. Frequently-accessed patient cache (fixed array) --
-- 8. Persistence (CSV save / load, failure handling) --
-- 9. Reports --
-- 10. Statistics and polymorphic description --
-- 11. String / date utilities --
-- 12. Identifier generator --

============================================
Passed: 279   Failed: 0
RESULT: ALL CHECKS PASSED
============================================
```

The qmake and CMake builds both complete with zero warnings. The GUI was launched, the seven tabs were rendered, and Windows UI Automation pressed the real Add Patient button with an empty form, producing the modal message “Patient name must be between 2 and 60 characters.” after which the application remained usable.

28 Data File Formats

The clinic is stored as three CSV files in the selected data folder. Comment lines start with `#`, fields are quoted when necessary, and quoted quotes are doubled.

```
# patients.csv
type,id,name,age,gender,contact,registeredOn,field1,field2,field3
GENERAL,P-0001,Amina Bello,34,Female,+234 803 555 0101,2026-10-21,NHIS,Hypertension
EMERGENCY,P-0002,Chinedu Okafor,41,Male,+234 805 555 0102,2026-10-21,2,Ambulance,yes
FOLLOWUP,P-0003,Fatima Yusuf,28,Female,+234 807 555 0103,2026-10-21,2026-09-21,14,Dr. Adeyemi

# appointments.csv
id,patientId,date,time,serviceCode,clinician,status,fee,notes
APT-0001,P-0001,2026-10-14,09:00,GEN-CON,Dr. Adeyemi,Completed,2500.00,Blood pressure review
```

The seed dataset contains 14 patients across the three categories, 25 appointments spread around the current date, and a service catalogue of twelve medical services.

Submission Checklist

Required item	Where it is
Complete labelled source code	GitHub repository and Part 4
Screenshots (dashboard, registration, appointments, search/sort, queue, reports)	Part 3, Figures 3-9
UML / class diagram	Part 2, Figures 1-2
Project report with the twelve required sections	Part 1
Data files required to run the application	`data/` in the repository

All source files carry a header comment naming their layer and purpose, and the repository history preserves the implementation together with its tests and documentation.

Declaration. I confirm that this submission is my own work, that the searching, sorting and recursive algorithms were implemented by me rather than replaced with library calls, and that the only external framework used is Qt 6 with the C++ standard library.

End of submission.